

8.1 Concepto de Clase e Instanciación en Dart

Una clase en Dart es una plantilla o plano (*blueprint*) que define las propiedades (atributos) y comportamientos (métodos) de una entidad. Dart es un lenguaje puramente orientado a objetos: todos los valores son instancias de alguna clase que hereda directa o indirectamente de `Object`.

A diferencia de lenguajes tradicionales como Java o C++, en Dart **no se requiere la palabra reservada `new`** para instanciar una clase (aunque es sintácticamente válida, su uso se considera obsoleto desde Dart 2.0).

```
class Alumno {
  String boleta = '';
  String nombre = '';

  void estudiar() {
    print('$nombre está estudiando.');
```

8.2 Constructores: Declarativos, Nombrados y Constantes

El constructor es el método especial encargado de inicializar el estado del objeto al momento de su creación. Dart provee diferentes tipos de constructores optimizados para resolver necesidades arquitectónicas específicas:

1. Constructor Generativo con *Initializing Formal Parameters*

Dart permite asignar parámetros directamente a los campos de la clase en la misma firma del constructor utilizando la sintaxis `this.propiedad`, eliminando el código repetitivo de asignación manual dentro del cuerpo del constructor:

```
class Profesor {
  String nombre;
  String academia;

  // Asignación directa e inmutable en firma
  Profesor(this.nombre, this.academia);
```

```
}
```

2. Constructores Nombrados (*Named Constructors*)

Permiten definir múltiples formas de instanciar una misma clase bajo nombres semánticos claros (muy útil para deserializar JSONs de APIs REST):

```
class Usuario {  
  String id;  
  String email;  
  
  Usuario(this.id, this.email);  
  
  // Constructor nombrado para la ingesta desde un Map/JSON  
  Usuario.fromJson(Map<String, dynamic> json)  
    : id = json['id'] as String,  
      email = json['email'] as String;  
}
```

3. Constructores Constantes (*const*)

Si una clase produce objetos cuyos campos son inmutables (*final*), podemos definir un constructor *const*. Esto le indica al compilador de Dart que canonicamente la instancia en memoria durante el tiempo de compilación, elevando significativamente el rendimiento en renderizados de Flutter.

8.3 Encapsulamiento, Getters y Setters Explícitos

En Dart **no** existen las palabras clave *private*, *protected* o *public*. El encapsulamiento no opera a nivel de clase, sino a nivel de **Librería (Archivo)**.

- Para marcar un atributo o método como privado, se le antepone un guion bajo (*_*).
- Los miembros precedidos por *_* solo son accesibles desde el código escrito dentro del mismo archivo *.dart*.

Getters y Setters

Permiten controlar la lectura y escritura de los atributos privados mediante la aplicación de validaciones, transformaciones o lógica de negocio intermediaria:

```
class CuentaEscolar {  
  double _promedio = 0.0; // Atributo privado al archivo  
  
  // Getter explícito  
  double get promedio => _promedio;
```

```
// Setter explícito con validación de fronteras
set promedio(double nuevoPromedio) {
  if (nuevoPromedio >= 0.0 && nuevoPromedio <= 10.0) {
    _promedio = nuevoPromedio;
  } else {
    print('Error: El promedio $nuevoPromedio está fuera de rango (0.0 - 10.0).');
  }
}
}
```

8.4 Ejemplo

Creen el archivo `bin/d8_clases_getters_setters.dart` en VS Code. El siguiente programa demuestra la creación de una clase con campos privados, constructores nombrados, constructores constantes y validación mediante getters/setters.

```
/// Módulo D8: Clases, Constructores, Getters y Setters
/// Demostración práctica de encapsulamiento, constructores nombrados y validación de datos.

void main() {
  print('=====');
  print(' SISTEMA DE GESTIÓN DE EXPEDIENTES (POO DART) - ESCOM ');
  print('=====');

  // 1. Instanciación con constructor primario
  print('--- 1. Instanciación Tradicional ---');
  final alumno1 = EstudianteEscom(
    boleta: '2021630001',
    nombre: 'Ana Martínez',
    creditosAcumulados: 120,
  );

  // Intentamos asignar una calificación mediante el setter validado
  alumno1.promedio = 9.4; // Válido
  alumno1.promedio = 11.5; // Rechazado por el setter

  print('Ficha: ${alumno1.fichaInformativa}');
  print('Estatus de Dictamen: ${alumno1.estatusDictamen}');

  // 2. Instanciación con Constructor Nombrado (.fromJson)
  print('--- 2. Instanciación desde un Payload JSON ---');
  final Map<String, dynamic> payloadApi = {
    'boleta': '2021630099',
    'nombre': 'Carlos Mendoza',
    'creditos': 280,
    'promedioInicial': 5.8,
```

```

};

final alumno2 = EstudianteEscom.fromJson(payloadApi);
print('Ficha API: ${alumno2.fichaInformativa}');
print('Estatus de Dictamen: ${alumno2.estatusDictamen}\n');

// 3. Demostración de Constructor Constante (Canonicalización en memoria)
print('--- 3. Verificación de Instancias Constantes ---');
const config1 = ConfiguracionSistema(maxAlumnosPorGrupo: 40, ambiente: 'PROD');
const config2 = ConfiguracionSistema(maxAlumnosPorGrupo: 40, ambiente: 'PROD');

// Comparación de identidades en memoria con identical()
print('¿Son config1 y config2 la misma instancia física en memoria?');
print(' -> ${identical(config1, config2) ? "Sí (Misma memoria canónica)" :
"NO"}\n');

print('=====');
}

/// Clase que representa la entidad de un estudiante en la ESCOM.
class EstudianteEscom {
  // Atributos públicos e inmutables
  final String boleta;
  final String nombre;
  final int creditosAcumulados;

  // Atributo privado encapsulado (prefijado con _)
  double _promedio = 0.0;

  // Constructor generativo principal con asignación de parámetros nombrados
  EstudianteEscom({
    required this.boleta,
    required this.nombre,
    required this.creditosAcumulados,
  });

  // Constructor nombrado para deserializar mapas/JSONs con lista de
  inicialización
  EstudianteEscom.fromJson(Map<String, dynamic> json)
    : boleta = json['boleta'] as String,
      nombre = json['nombre'] as String,
      creditosAcumulados = json['creditos'] as int {
    // Inicialización del promedio protegido dentro del cuerpo
    final double? promedioParsed = (json['promedioInicial'] as num?)?.toDouble();
    if (promedioParsed != null) {
      promedio = promedioParsed; // Hace uso del setter validado
    }
  }

  // Getter explícito para consultar el promedio privado
  double get promedio => _promedio;

  // Setter explícito con validación de reglas de negocio
  set promedio(double nuevoPromedio) {

```

```
    if (nuevoPromedio >= 0.0 && nuevoPromedio <= 10.0) {
      _promedio = nuevoPromedio;
    } else {
      print(' [ERROR DE VALIDACIÓN] Promedio $nuevoPromedio fuera de rango (0.0 - 10.0). Asignación denegada.');
```

Instrucciones de Ejecución en VS Code:

```
dart run bin/d8_clases_getters_setters.dart
```

8.5 Ejercicios Prácticos

Implementen los siguientes dos programas en su entorno local para dominar la encapsulación, validaciones mediante setters y constructores nombrados.

Ejercicio 1: Encapsulamiento y Control de Sensor IoT Mobile

Crea una clase llamada `SensorTemperatura`:

1. Define un atributo privado `_celsius` de tipo `double` inicializado en `0.0`.
2. Implementa un getter explícito `celsius` y un setter `celsius` que impida asignar temperaturas inferiores a `-273.15 °C` (Cero Absoluto). Si se intenta ingresar un valor menor, debe imprimir un mensaje de alerta y no actualizar el valor.

3. Agrega un getter calculado `fahrenheit` que transforme la temperatura privada a grados Fahrenheit usando la fórmula: $F = (C \times \frac{9}{5}) + 32$.
4. Instancia el sensor en `main()`, prueba la asignación de valores válidos e inválidos y muestra las lecturas convertidas.

Ejercicio 2: Modelo DTO de Producto de Tienda Escolar

Escribe una clase `ProductoApp`:

1. Atributos: `id` (`String`), `nombre` (`String`), `precio` (`double`), `stock` (`int`).
 2. Implementa el constructor generativo principal.
 3. Crea un constructor nombrado `ProductoApp.desdeJson(Map<String, dynamic> json)` para parsear instancias provenientes de un mapa.
 4. Define un setter para `stock` que evite que el inventario sea negativo.
 5. Agrega un getter calculado `estaDisponible` de tipo `bool` que devuelva `true` si `stock > 0`.
 6. Demuestra en `main()` la creación de un producto desde un Map, modifica su stock con valores válidos e inválidos e imprime su disponibilidad.
-